Day 3 – Services & Dependency Injection

🏷️ Setup & Prerequisites

1. **Setup Checklist**:
   - **Virtual Environment/Dev Server**: Ensure `ng serve` is running in your terminal.
   - **IDE Ready**: Open your project in VS Code, ready to generate and open new service scripts.
2. **Prerequisites**:
   - You must have completed Day 2 successfully (understanding component states, databinding, and template loop rendering).

🎯 Learning Objectives

By the end of this class, you will be able to:

- Explain the role of services in Angular for encapsulating business logic and data access.
- Describe how Angular's Dependency Injection (DI) system locates and registers services.
- Define services using the `@Injectable({ providedIn: 'root' })` decorator.
- Use RxJS `Observable` and the `of` operator to simulate asynchronous data flows.
- Inject a service into a component constructor and consume data streams via `.subscribe()`.

⏱️ Classroom Timeline (45 min)

Segment	Duration	Focus
1. Hook & Big Picture	7 min	Define separation of concerns. Contrast components (views) with services (data providers).
2. Key Concepts & Core Ideas	7 min	Explain Dependency Injection tree lifecycle, RxJS Observables, and Subscription streams.
3. Live Coding Walkthrough	23 min	Generate services, implement Observable data fetches, inject into components, and subscribe to data.

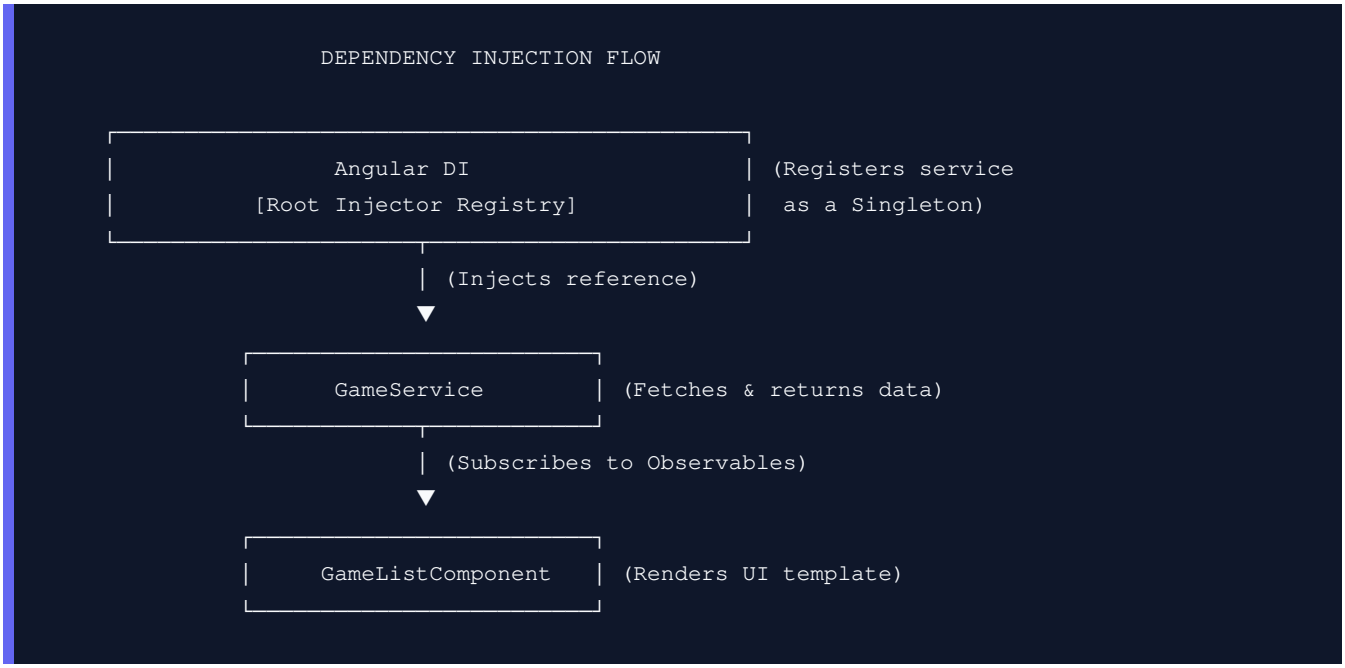
4. Check for Understanding	5 min	Q&A on Singleton services, providedIn metadata, and subscribing leaks.
5. Class Wrap-up & Checkpoint	3 min	Verify data fetches populate component views successfully from the service.

🚀 Step-by-Step Walkthrough

1. Hook & Big Picture (7 min)

In standard components, keeping hardcoded arrays of data is a bad practice. If multiple components (like a shopping cart, a sidebar, and a checkout page) need to access the same list of games, duplicating code causes state synchronization bugs. We solve this by moving the data logic out of the component and into a centralized file called a **Service**.

- **Separation of Concerns**: The component's job is to look good and handle UI interaction (the view). The service's job is to fetch data and process business logic (the model). Angular joins them using Dependency Injection.



2. Key Concepts & Core Ideas (7 min)

Introduce these architecture concepts:

- **Angular Service**: A reusable class containing shared logic, helper functions, or network connection calls that can be injected across multiple components.
- **Dependency Injection (DI)**: A software design pattern where classes request dependencies (like

services) from an external injector rather than instantiating them manually using .

- `@Injectable()`: A decorator placed on service classes to register them with Angular's DI container.
- `providedIn: 'root'`: Registers the service as a **Singleton** globally. Angular creates only ONE instance of this service for the entire application lifecycle, saving memory and keeping state synced.
- **RxJS Observable**: A representation of a stream of data that can be observed asynchronously. Unlike a Promise which returns once, an Observable can emit multiple values over time.
- **Subscription (`.subscribe()`)**: Components trigger execution of Observables and listen for emitted data streams by calling `.subscribe()`.

---

3. Live Coding Walkthrough (23 min)

Step 3.1: Creating an Angular Service

- **Concept**: Generate a new service called `GameService` using the Angular CLI.
- **Code**:

```
# Generate a service under src/app/services/game.service.ts
ng generate service services/game
```

- **Verify**: Check that `src/app/services/` contains `game.service.ts` and `game.service.spec.ts` files.

Step 3.2: Implementing Service Observable Logic

- **Concept**: Open `src/app/services/game.service.ts`. We will create a list of games, and define methods that return RxJS Observables to simulate fetching data from a database.
- **Code**: Modify `src/app/services/game.service.ts`:

```
import { Injectable } from '@angular/core';
import { Observable, of } from 'rxjs'; // Import RxJS utilities

export interface Game {
  id: number;
  title: string;
  price: number;
  available: boolean;
}

@Injectable({
  // providedIn: 'root' makes the service globally accessible as a Singleton
  providedIn: 'root'
})
export class GameService {
  private games: Game[] = [
    { id: 1, title: 'Half-Life 3', price: 29.99, available: true },
    { id: 2, title: 'Cyberpunk 2077', price: 49.99, available: false },
    { id: 3, title: 'Portal 3', price: 19.99, available: true }
```

```
  ];

  constructor() {}

  # Method to fetch games list as an Observable stream
  # of() is an RxJS creator function that converts static data into an Observable stream
  getGames(): Observable<Game[]> {
    return of(this.games);
  }

  # Method to append a new game to the local inventory list
  addGame(title: string, price: number): Observable<Game> {
    const newGame: Game = {
      id: this.games.length + 1,
      title: title,
      price: price,
      available: true
    };
    this.games.push(newGame);
    return of(newGame);
  }
}
```

Step 3.3: Injecting and Consuming the Service in a Component

- **Concept**: Modify `GameListComponent` to inject `GameService` and retrieve the data stream inside the lifecycle hook `ngOnInit()`.
- **Code**: Modify `src/app/components/game-list/game-list.component.ts`:

```
import { Component, OnInit } from '@angular/core'; // Import OnInit lifecycle hook
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { GameService, Game } from '../../services/game.service'; // Import service class

@Component({
  selector: 'app-game-list',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './game-list.component.html',
  styleUrls: ['./game-list.component.css']
})
export class GameListComponent implements OnInit {
  newGameTitle: string = '';
  newGamePrice: number = 0;

  // Define an empty array that will hold our fetched games
  games: Game[] = [];

  // Injecting the service in the class constructor
  // private keyword automatically creates a local class variable 'gameService'
  constructor(private gameService: GameService) {}
```

```
  // ngOnInit runs automatically once the component is initialized in the DOM
  ngOnInit(): void {
    this.fetchGames();
  }

  // Call service getGames Observable and subscribe to the output stream
  fetchGames(): void {
    this.gameService.getGames().subscribe({
      next: (data: Game[]) => {
        this.games = data;
      },
      error: (err) => {
        console.error('Error fetching games: ', err);
      }
    });
  }

  addGame(): void {
    if (this.newGameTitle.trim() === '') return;

    this.gameService.addGame(this.newGameTitle, this.newGamePrice).subscribe({
      next: (newGame: Game) => {
        // Re-fetch list to update UI state
        this.fetchGames();
        this.newGameTitle = '';
        this.newGamePrice = 0;
      }
    });
  }

  toggleAvailability(game: Game): void {
    game.available = !game.available;
  }
}
```

- **Verify**: Open `http://localhost:4200/` and check that the list is populated successfully. The user interface remains identical, but the state data is now safely isolated inside the service.

---

4. Check for Understanding (5 min)

> ***Question***: *"Why shouldn't we instantiate services using the `new` keyword? e.g. `this.gameService = new GameService()`?"*
> ***Answer***: *Creating instances manually defeats the purpose of Dependency Injection. If you do `new GameService()`, you create a new, separate instance of the service inside that component. If another component does the same, they will not share state (violating the Singleton pattern), and configuring mock services during unit testing becomes extremely difficult.*

> ***Question***: *"What is the difference between an Observable and a Promise?"*
> ***Answer***: *A Promise represents a single future value (either resolved or rejected) and executes immediately when created. An Observable represents a stream of values over time, can emit zero, one, or multiple values, and does not execute until a client calls `.subscribe()` (it is "cold" or lazy).*

> ***Question***: *"What does `@Injectable({ providedIn: 'root' })` do under the hood?"*
> ***Answer***: *It tells Angular's compiler to register this class with the application's root dependency injector. This creates a singleton service instance that is lazily loaded (created only when first requested by a component) and persists throughout the lifetime of the application.*

---

5. Daily Checkpoint & Wrap-up (3 min)

- **Verification Criteria**:
  - ○ The `GameService` is created and registered globally in root.
  - ○ The `GameListComponent` fetches data asynchronously using `.subscribe()` on `GameService.getGames()`.
  - ○ The component compiles and renders without errors.